Experiment 1

Title

Install XAMPP / WAMP. Install Git and set up a new repository. Learn fundamental Git commands: add, commit, modify, view. Create and manage branches for version control.

Theory:

Local Server Environment (XAMPP / WAMP)

Web applications often require a server, database, and scripting language to function properly. Installing a live server for development and testing is neither economical nor practical for beginners. To overcome this limitation, local server environments such as XAMPP and WAMP are used.

XAMPP is a cross-platform software package that provides Apache web server, MySQL (or MariaDB) database, PHP, and Perl in a single installation. It allows developers to host and test dynamic web applications locally. WAMP performs the same function but is limited to Windows operating systems.

These tools simulate a real hosting environment on a personal computer, enabling students to understand client-server architecture, server-side scripting, and database interaction without an internet connection.

Need for Version Control

During software development, source code is modified frequently. Managing multiple versions manually can lead to data loss, overwriting of files, and confusion. A Version Control System (VCS) solves these problems by maintaining a complete history of changes made to files over time.

Version control allows developers to:

- Track who made changes and when
- Restore previous versions of files
- Work collaboratively on the same project
- Maintain stable and experimental versions simultaneously

Types of Version Control Systems

There are two major types of version control systems:

Centralized Version Control System (CVCS)

In this system, a single central server stores the repository, and all users interact with it. Examples include SVN and CVS. If the central server fails, the entire system becomes unavailable.

Distributed Version Control System (DVCS)

In a distributed system, each user has a complete copy of the repository, including full history. Git is a distributed version control system, making it faster, more reliable, and suitable for collaborative development.

Git as a Distributed Version Control System

Git is an open-source distributed version control system developed by Linus Torvalds. Unlike centralized systems, Git allows developers to work offline and commit changes locally. Synchronization with remote repositories can be done later.

Git efficiently tracks changes using snapshots instead of differences, which improves performance and reliability. Each commit represents a complete state of the project at a given time.

Git Repository Structure

A Git repository consists of:

- Working Directory – where files are edited
- Staging Area – where changes are prepared for commit
- Repository (.git directory) – where commits and metadata are stored

The repository maintains:

- Commit history
- Branch information
- Configuration settings
- Object database

Git Workflow

The standard Git workflow involves three stages:

1. **Modify** – Changes are made in files
2. **Stage** – Changes are added to the staging area
3. Commit – Changes are saved permanently in the repository

This workflow ensures controlled and organized version tracking.

Fundamental Git Commands:

- **git init**
  Initializes a new Git repository in a directory.
- **git add**
  Adds modified or new files to the staging area.
- **git commit**
  Saves staged changes as a snapshot with a descriptive message.
- **git status**
  Displays the current state of the working directory and staging area.
- **git log**
  Shows the history of commits along with commit IDs, author details, and timestamps.

Branching in Git

A branch represents an independent line of development. The default branch is usually named main. Branching allows developers to work on new features, bug fixes, or experiments without affecting the stable version of the project.

Git branches are lightweight and efficient, making branch creation and deletion fast operations.

Branch Management

Branch management includes:

- Creating branches for specific tasks
- Switching between branches

- Merging branches into the main branch
- Deleting obsolete branches

This mechanism enables parallel development and improves productivity in team projects.

Advantages of Git in Software Development

A. Maintains complete project history

B. Enables collaboration among multiple developers

C. Prevents accidental data loss

D. Supports branching and merging

E. Allows rollback to earlier versions

F. Works efficiently with remote repositories such as GitHub

Prerequisites

- Windows/Linux OS
- Internet connection
- Basic command-line knowledge

Procedure

A. Install XAMPP/WAMP

1. Download XAMPP from the official Apache Friends website.
2. Run the installer and select Apache, MySQL, and PHP components.
3. Complete installation and open XAMPP Control Panel.
4. Start Apache and MySQL services. 5. Verify installation by opening http://localhost in a browser.

B. Install and Configure Git

1. Download Git from the official Git website.
2. Install Git with default settings.
3. Open Git Bash and configure user details: git config --global user.name "Your Name" git config --global user.email youremail@example.com

C. Create Repository and Perform Git Operations

1. Open Visual Studio Code on the system.
2. Create a new folder named myproject.
3. Open the folder in VS Code
   (File → Open Folder → select myproject).
4. Open the Terminal in VS Code (Terminal → New Terminal).
5. Initialize a new Git repository in the project folder:

git init

6. Create a new HTML file named index.html.
7. Write basic HTML code in index.html and save the file:
   </head>

   <body>

   <h1>Hello Git</h1>

   </body>

   </html>
8. Check the status of the repository:

   git status
9. Add the HTML file to the staging area: git add index.html
10. Commit the staged file with a commit message: git commit -m "Add index.html file"
11. Modify the HTML file by adding new content and save it:

    This file is tracked using Git.
12. Check the status to view modified files: git status
13. Add the modified file to the staging area: git add index.html
14. Commit the updated HTML file: git commit -m "Update index.html content"
15. View the commit history:

    git log

Branch Creation and Management:

1. Create a new branch named feature git branch feature
2. Switch to the new branch: git checkout feature
3. Modify index.html by adding new content and save:
   This content is from feature branch.
4. Add and commit changes in the feature branch: git add

   index.html git commit -m "Update index.html in feature

   branch"
5. Switch back to the main branch: git checkout main

   <!DOCTYPE html>

   <html>

   <head>

   <title>My First Git Project</title>
6. Merge the feature branch into main: git merge feature
7. View all branches:

git branch

Push Local Repository to Github.

1. Create a new repository on GitHub (without README).
2. Add GitHub repository as remote origin:

   git remote add origin https://github.com/username/repository-name.git
3. Push local main branch to GitHub:

   git branch -M main git push -u origin

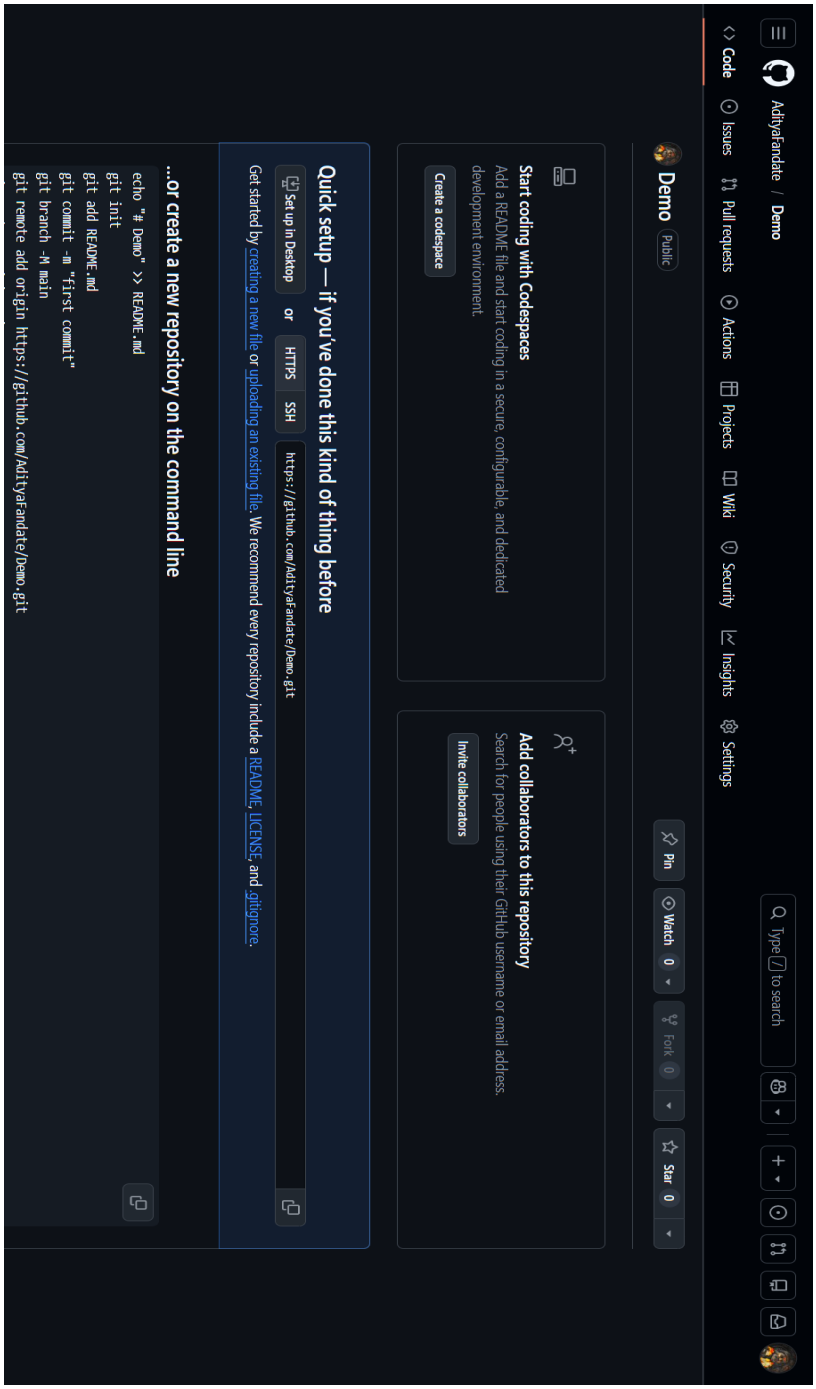
   main

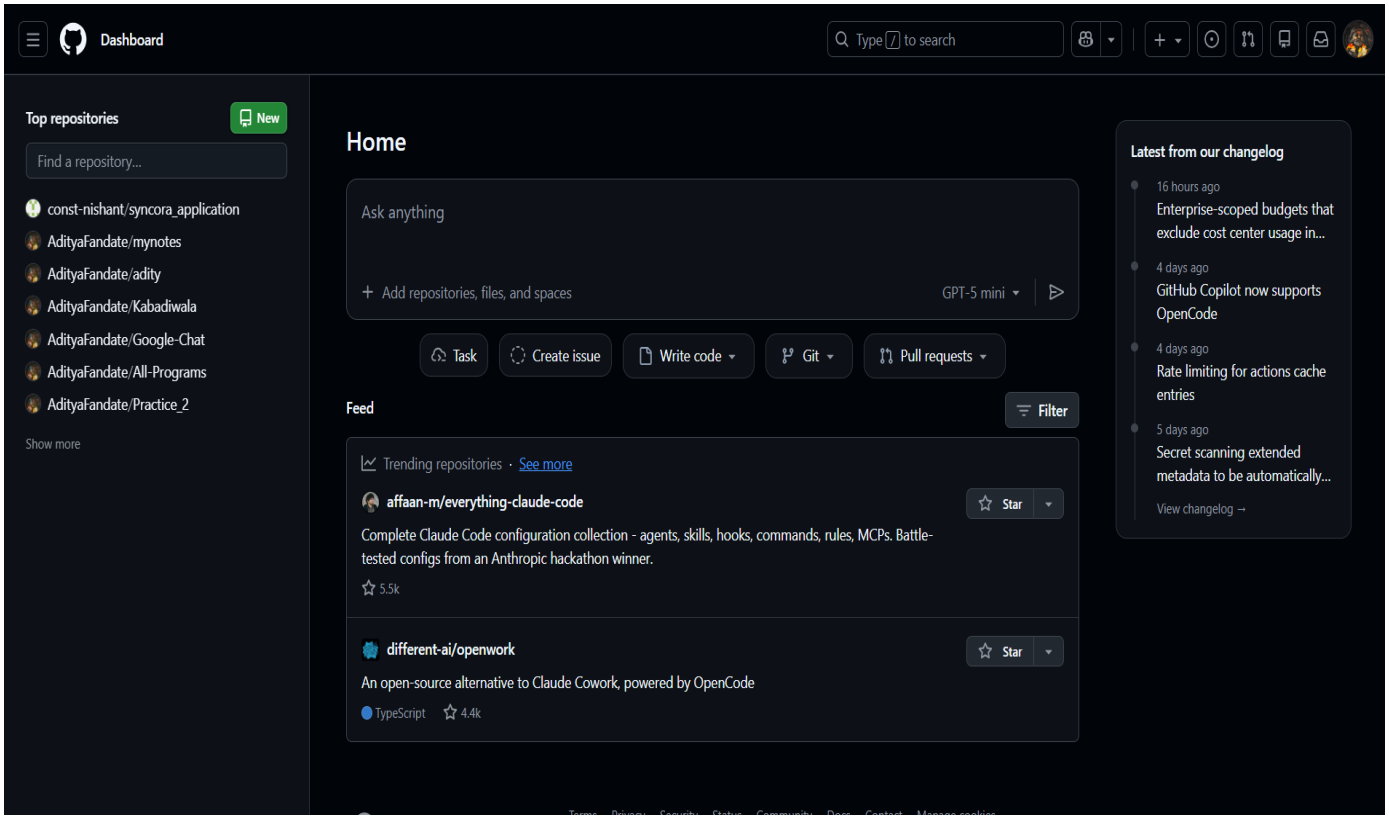
Pull Changes From Github:

1. Make any change directly on GitHub (or from another system).
2. Pull latest changes into local repository: git pull origin main
3. View Commit History

   git log

Output:


AdityaFandate / Demo
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Demo
Public
Pin
Watch 0
Fork 0
Star 0
Start coding with Codespaces
Add a README file and start coding in a secure, configurable, and dedicated development environment.
Create a codespace
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
Set up in Desktop
or
HTTPS
SSH
https://github.com/AdityaFandate/Demo.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# Demo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/AdityaFandate/Demo.git


Dashboard
Type / to search
Top repositories
New
Find a repository...
const-nishant/syncora_application
AdityaFandate/mynotes
AdityaFandate/adity
AdityaFandate/Kabadiwala
AdityaFandate/Google-Chat
AdityaFandate/All-Programs
AdityaFandate/Practice_2
Show more
Home
Ask anything
Add repositories, files, and spaces
GPT-5 mini
Task
Create issue
Write code
Git
Pull requests
Feed
Filter
Trending repositories · See more
affaan-m/everything-claude-code
Star
Complete Claude Code configuration collection - agents, skills, hooks, commands, rules, MCPs. Battle-tested configs from an Anthropic hackathon winner.
5.5k
different-ai/openwork
Star
An open-source alternative to Claude Cowork, powered by OpenCode
TypeScript
4.4k
Latest from our changelog
16 hours ago
Enterprise-scoped budgets that exclude cost center usage in...
4 days ago
GitHub Copilot now supports OpenCode
4 days ago
Rate limiting for actions cache entries
5 days ago
Secret scanning extended metadata to be automatically...
View changelog →

MINGW64:/c/Users/adity

```
adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git --version
git version 2.45.2.windows.1

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ cd desktop
bash: cd: desktop: No such file or directory

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ mkdir Github

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git init
Reinitialized existing Git repository in C:/Users/adity/.git/

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ touch README.md

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ My first Git Repo
bash: My: command not found

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ cat > README.md
My first Git Repo



adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master
```

```
adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ cat README.md
My first Git Repo


adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the nex
t time Git touches it

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

MINGW64:/c/Users/adity

```
adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git commit -m "initail commit"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'adity@LAPTOP-9U1VSQ45.(none)')

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git config -- global user.name "Aditya"
error: key does not contain a section: global

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git config --global user.name "Aditya"

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git config --global user.name "adityafandate@gmail.com"

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git config --global user.name "Aditya"

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git config --global user.email "adityafandate007@gmail.com"

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git config --global --list
user.name=Aditya
user.email=adityafandate007@gmail.com

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git commit -m "initail commit"
[master (root-commit) 7fea825] initail commit
 1 file changed, 2 insertions(+)
 create mode 100644 README.md

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git commit -m main
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master
```

```
MINGW64:/c/Users/adity

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git -branch -m main
unknown option: -branch
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
           [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
           [--config-env=<name>=<envvar>] <command> [<args>]

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (master)
$ git branch -m main

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$ git branch
* main

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$ * main
bash: AppData: command not found
```

```
adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$ git log --online
fatal: unrecognized argument: --online

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$ git log --oneline
7fea825 (HEAD -> main) initail commit

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$
```


```
adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$ git log --online
fatal: unrecognized argument: --online

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$ git log --oneline
7fea825 (HEAD -> main) initail commit

adity@LAPTOP-9U1VSQ45 MINGW64 ~ (main)
$
```


Conclusion:

Thus, XAMPP/WAMP and Git were installed successfully. Basic Git commands were executed, and branch management was performed to understand version control operations.

Questions:

1.What is a version control system?

A **Version Control System (VCS)** is a tool that helps track changes in files (mainly code) over time. It allows multiple people to work on the same project without conflicts.

Key features:

- Keeps history of changes
- Allows reverting to previous versions
- Supports team collaboration
- Tracks who made what changes

Example: Git

2. What is branching in Git?

Branching in Git means creating a separate line of development from the main project.

- The default branch is usually called **main** or **master**
- Developers create branches to:
  - Work on new features
  - Fix bugs
  - Experiment safely

Example:

- `main` → stable code
- `feature-login` → new login feature

After completing work, branches are merged back into the main branch.

3. What is GitHub and how is it used for collaboration?

GitHub is a cloud-based platform that hosts Git repositories.

How it helps collaboration:

- Developers can **upload (push)** code to GitHub
- Others can **download (clone/pull)** code
- Teams can work together using:
  - **Pull Requests (PRs)** → propose changes
  - **Code Reviews** → check code quality
  - **Issues** → track bugs/tasks
- Keeps everything centralized and accessible online

4. Difference between local repository and remote repository.

Feature	Local Repository	Remote Repository
Location	On your computer	On a server (e.g., GitHub)
Access	Only you	Team members can access
Speed	Faster (offline work)	Slower (needs internet)
Purpose	Development & testing	Sharing & collaboration
Example	Git folder on your PC	GitHub repository